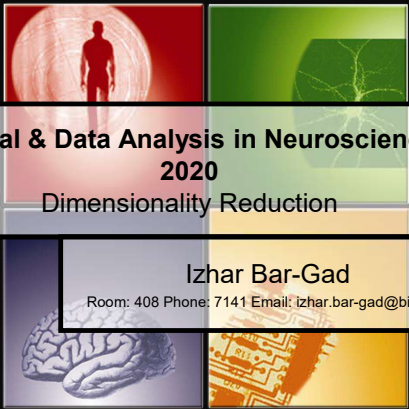



**Signal & Data Analysis in Neuroscience
2020**

Dimensionality Reduction

Izhar Bar-Gad
Room: 408 Phone: 7141 Email: izhar.bar-gad@biu.ac.il

1



Material disclaimer

- The presentation uses figures and slides taken from the following tutorials:
- A tutorial on Principal Component Analysis / Jonathon Shlens
- Principal Component Analysis / Ricardo Gutierrez-Osuna
- A tutorial on Principal Components analysis / Lindsay Smith
- Unsupervised learning - Kernel PCA / Rita Osadchy

IBG

2



Overview

- **Dimensionality reduction**
- Principal component analysis (PCA)
- Extending PCA
- Beyond PCA

IBG

3






Curse of dimensionality I

- Data in many domains is complex and made up of many observed variables (**dimensions**) of interest.
- Curse of dimensionality** (Richard Bellman 1961) - the number of **samples** per variable increase **exponentially** with the number of variables to maintain a given level of **accuracy**.
- In many realistic scenarios the number of samples is inadequate to estimate the **multivariate** distribution.

4





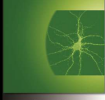

Curse of dimensionality example I

- Consider a 3-class pattern recognition problem
 - A simple approach would be to
 - Divide the feature space into uniform bins
 - Compute the ratio of examples for each class at each bin and,
 - For a new example, find its bin and choose the predominant class in that bin
 - In our toy problem we decide to start with one single feature and divide the real line into 3 segments


- After doing this, we notice that there exists too much overlap among the classes, so we decide to incorporate a second feature to try and improve separability

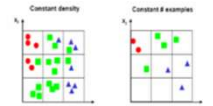
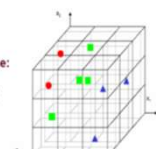
5



Curse of dimensionality example II

- We decide to preserve the granularity of each axis, which raises the number of bins from 3 (in 1D) to $3^2=9$ (in 2D)
 - At this point we need to make a decision: do we maintain the density of examples per bin or do we keep the number of examples had for the one-dimensional case?
 - Choosing to maintain the density increases the number of examples from 9 (in 1D) to 27 (in 2D)
 - Choosing to maintain the number of examples results in a 2D scatter plot that is very sparse
- Moving to three features makes the problem worse:
 - The number of bins grows to $3^3=27$
 - For the same density of examples the number of needed examples becomes 81
 - For the same number of examples, well, the 3D scatter plot is almost empty

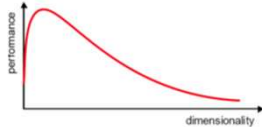



6



Curse of dimensionality II

- For a given sample size, there is a maximum number of features above which the performance of a classifier will degrade rather than improve
- In most cases, the additional information that is lost by discarding some features is (more than) compensated by a more accurate mapping in the lower dimensional space



7



Dimensionality reduction I

- Feature selection** – choosing the most informative features.

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{feature selection}} \begin{bmatrix} x_{i_1} \\ x_{i_2} \\ \vdots \\ x_{i_l} \end{bmatrix}$$

- Feature extraction** – creating new features by combining existing ones.

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{feature extraction}} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = f \left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \right)$$

8

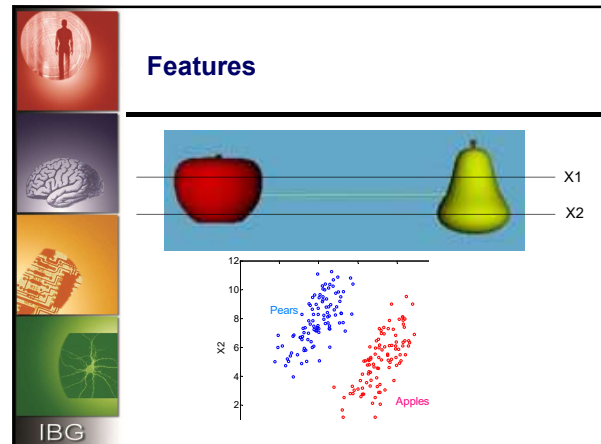


Dimensionality reduction II

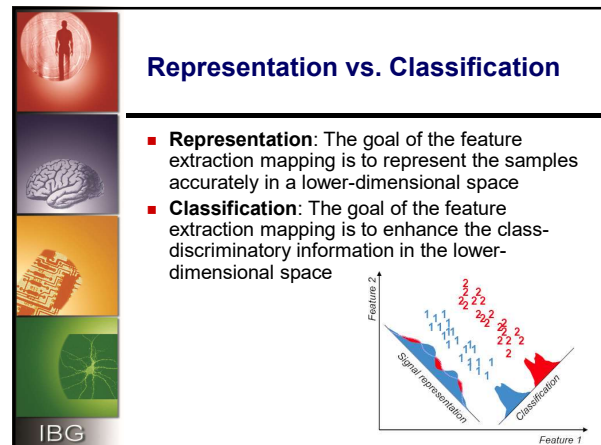
- In general the optimal mapping $y=f(x)$ is non-linear.
- However, since there is no systematic way to generate **non-linear transforms** we will limit this lecture to **linear transforms**.

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{linear feature extraction}} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1N} \\ w_{21} & w_{22} & \cdots & w_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ w_{M1} & w_{M2} & \cdots & w_{MN} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$$

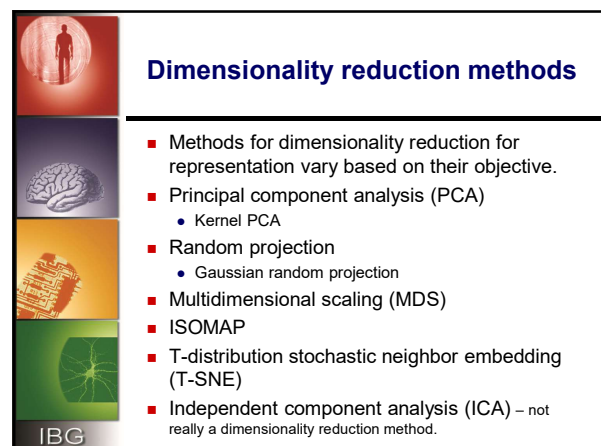
9



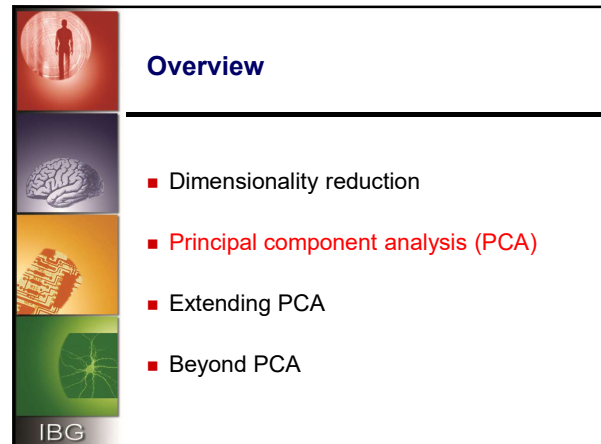
10



11



12

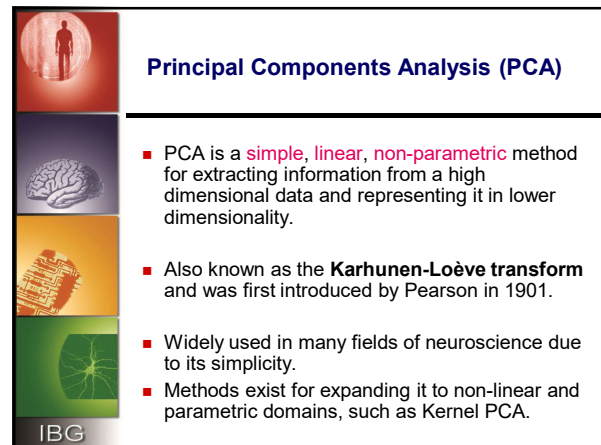


Overview

- Dimensionality reduction
- **Principal component analysis (PCA)**
- Extending PCA
- Beyond PCA

IBG

13

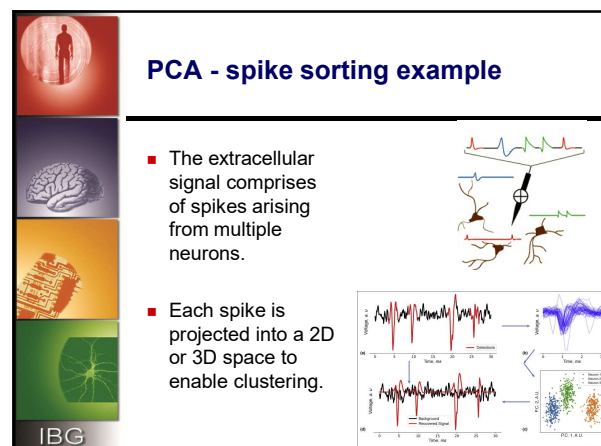


Principal Components Analysis (PCA)

- PCA is a **simple, linear, non-parametric** method for extracting information from a high dimensional data and representing it in lower dimensionality.
- Also known as the **Karhunen-Loève transform** and was first introduced by Pearson in 1901.
- Widely used in many fields of neuroscience due to its simplicity.
- Methods exist for expanding it to non-linear and parametric domains, such as Kernel PCA.

IBG

14



PCA - spike sorting example

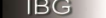
- The extracellular signal comprises of spikes arising from multiple neurons.
- Each spike is projected into a 2D or 3D space to enable clustering.

IBG

15

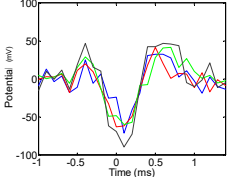






PCA – spike sorting example

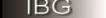
- Spikes are recorded for 2.5ms period in bins of 0.1ms leading to samples of 25 points.
- Recording of multiple spikes leads to data in 25 dimensions.



16

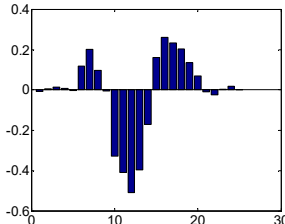






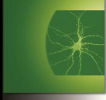
PCA – spike sorting example

- Are these spikes from a single neuron? What are the difference between spikes?
- The first PC forms the following axis:



17




Change of basis

- Is there a different basis, which is a linear combination of the original basis that **best** re-expresses our data?
- Naïve basis → different orthonormal basis.

k – samples n - variables
 X – Original data ($n \times k$)
 Y – Transformed data ($n \times k$)
 P – Transformation matrix ($n \times n$)

$$PX = Y$$

18






Which basis?

- What is the best way to “re-express” X?
- What is the best choice for the basis P?



19

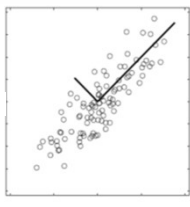


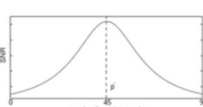



Signal to noise ratio

Maximize the signal to noise ratio (SNR or S/N)

$$SNR = \frac{\sigma_{Signal}^2}{\sigma_{Noise}^2}$$

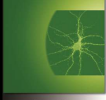






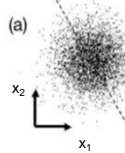
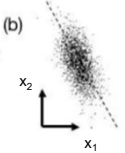
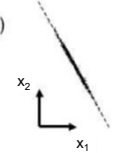
20



Redundancy

- The variables (dimensions) vary from non-correlated (a) to highly correlated (c)

low redundancy

high redundancy



21






Variance & Covariance

- **Variance** – measurement of the spread (or dispersion) of a variable around its mean.
- **Covariance** – measurement of the spread of one variable in relation to another variable.
- For $x=\{x_1, x_2, \dots, x_k\}$ $y=\{y_1, y_2, \dots, y_k\}$

$$Var(x) = \frac{\sum_{i=1}^k (x_i - \bar{x})^2}{k} \quad Cov(x, y) = \frac{\sum_{i=1}^k (x_i - \bar{x})(y_i - \bar{y})}{k}$$

* For samples, the unbiased estimator has a denominator of (k-1)

22






Covariance matrix

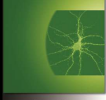
- We can generalize the covariance to n variables $X=\{x_1, x_2, \dots, x_n\}$ over k observations to form the $(n \times k)$ observations matrix

$$Cov(X) = \frac{XX^T}{k-1}$$

- C_x is symmetric and its diagonal values are the variances of the variables.
- High diagonal values indicate interesting dynamics.
- High off-diagonal values indicate high redundancy.

23



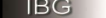
Best basis...

- Thus our goal is to:
 - Maximize the signal (variance)
 - Minimize the noise (covariance)
- This optimization reflects in zeros in the off-diagonal values of C_y
- PCA is a simple method of achieving this diagonalization using the orthonormal (i.e. $p_i p_j = \delta_{ij}$) basis

24







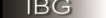
Naïve PCA algorithms

1. Define p_1 as the normalized direction in n dimensional space in which the variance of X is maximized.
2. Define p_i as the normalized direction with maximal variance in a direction perpendicular to all previously selected p 's
3. Repeat stage 2 until all n vectors are found.

25





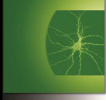


Assumptions and limits

- Linearity
 - Linearity frames the problem as a change of basis (non-linear extensions exist with kernel-PCA, ICA...)
- Mean and variance are sufficient statistics
 - The only class of probability distributions that are described fully by their first two moments are the exponential distribution (normal, exponential, ...)
- Large variances are important
- Principal components are orthogonal

26




Solving PCA – algebraic solution

- Select P to be a matrix in which each row p_i is an eigenvector of XX^T
- The principal components of X may be calculated by deriving the eigenvalues & eigenvectors of XX^T
- The i^{th} diagonal value of C_Y is the variance of X along p_i
- By rearranging the diagonal according to its values we get the ordered principal components.

27



Eigenvalues & Eigenvectors

$$\begin{aligned}
 Var(y_k) &= \langle y_k^2 \rangle - \langle y_k \rangle^2 = \langle y_k^2 \rangle \\
 &= \left\langle \left(\sum_j x_j u_j^k \right)^2 \right\rangle = \left\langle \sum_{j,i} x_j x_i u_j^k u_i^k \right\rangle \\
 &= \sum_{j,i} u_j^k \langle x_j x_i \rangle u_i^k = \sum_{j,i} u_j^k c_{ji} u_i^k \\
 &= (\vec{U}^k)^T C \vec{U}^k = (\vec{U}^k)^T \lambda_k \vec{U}^k = \lambda_k (\vec{U}^k)^T \vec{U}^k = \lambda_k
 \end{aligned}$$

- The variance along the k^{th} eigenvectors in the k^{th} eigenvalue. Thus, most of the information in the data lies along the eigenvectors with the highest eigenvalues.

28



Solving PCA – algebraic solution

- Another method of calculating the principal components involves singular value decomposition (SVD).
- See: www.uwlab.edu/faculty/wil/svd/index.html
- SVD can be applied to any $m \times n$ matrix. While eigenvalue decomposition, on the other hand, can only be applied to (certain classes of) square matrices.
- In the special case of a Hermitian matrix which is positive semi-definite, i.e., all its eigenvalues are real and non-negative, then the singular values and singular vectors coincide with the eigenvalues and eigenvectors.
- Hermitian matrix - square matrix with complex entries which is equal to its own conjugate transpose $\rightarrow a_{ij} = a_{ji}^*$

29



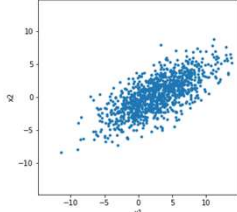
```

# Generate the 2D dataset
K = 1000
x1 = np.random.normal(loc=1, scale=4, size=K)
x2 = 0.5 * x1 + np.random.normal(loc=1, scale=2, size=K)
X = np.concatenate((x1.reshape(-1,1), x2.reshape(-1,1)), axis=1)

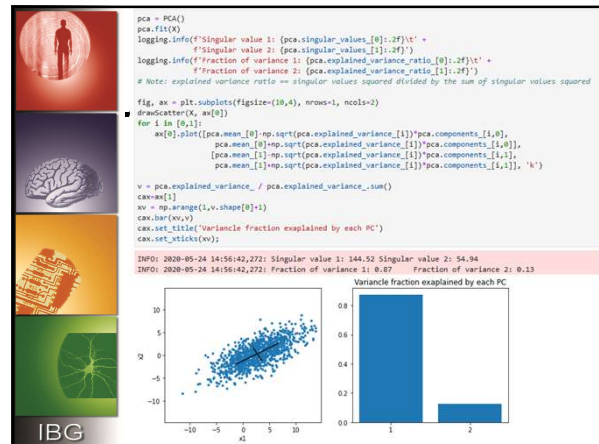
def drawScatter(X, axis):
    axis.plot(X[:,0], X[:,1], '.')
    axis.set_xlabel('x1')
    axis.set_ylabel('x2')
    ext = max((abs(X.min()), abs(X.max()))) + 1
    axis.set_xlim(-ext, ext)
    axis.set_ylim(-ext, ext)

fig, ax = plt.subplots(figsize=(5,5), nrows=1, ncols=1)
drawScatter(X, ax)
plt.savefig(os.path.join("figs", "SDA11-OriginalData.png"))

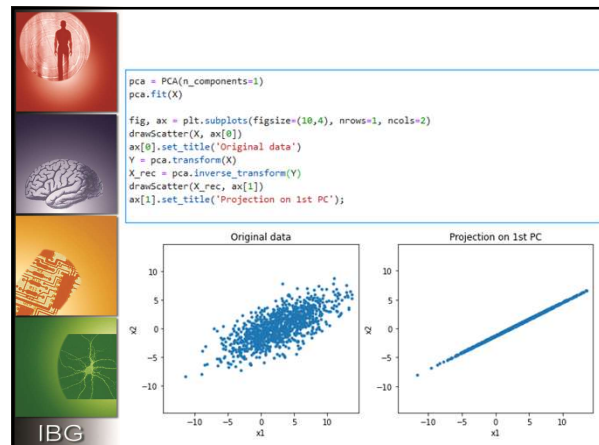
```



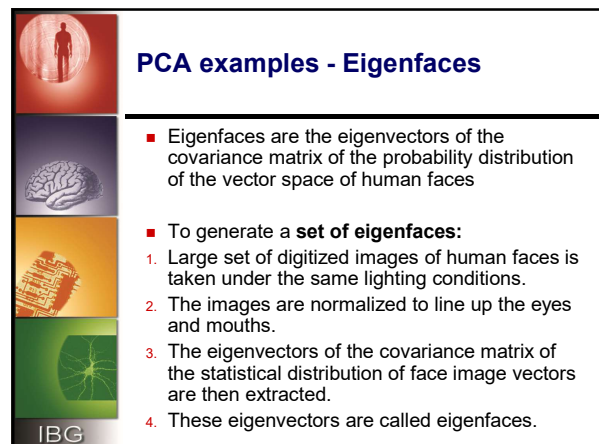
30



31



32



33






PCA examples - Eigenfaces

- the principal eigenface looks like a bland androgynous average human face






IBG <http://en.wikipedia.org/wiki/Image:Eigenfaces.png>

34





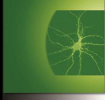

Eigenfaces – Face Recognition

- When properly weighted, eigenfaces can be summed together to create an approximate gray-scale rendering of a human face.
- Remarkably few eigenvector terms are needed to give a fair likeness of most people's faces
- Hence eigenfaces provide a means of applying data compression to faces for identification purposes.

IBG

35



Overview

- Dimensionality reduction
- Principal component analysis (PCA)
- Extending PCA
- Beyond PCA

IBG

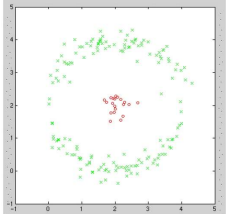
36



Non-linear PCA I

- PCA is limited to finding linear combinations of the dimensions.

→
Problem !!!
→

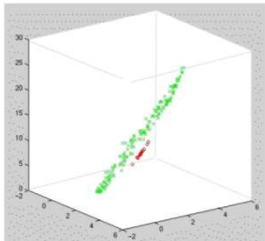


37



Non-linear PCA II

$$\Phi : \mathcal{X} = \mathbb{R}^2 \rightarrow \mathcal{H} = \mathbb{R}^3$$

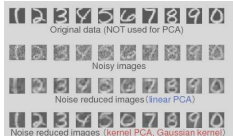
$$(x_1, x_2) \mapsto (x_1, x_2, x_1^2 + x_2^2)$$


38

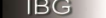


Kernel PCA

- Given a kernel $K(x_i, x_j) = \phi(x_i)^T \cdot \phi(x_j)$ the solution of a new space it simple.
- Note that the kernel size is $n \times n$ where n is the number of points.
- Gaussian kernel $K(x, x') = e^{-\beta \cdot \|x - x'\|^2}$



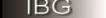
39



Overview

- Dimensionality reduction
- Principal component analysis (PCA)
- Extending PCA
- Beyond PCA

40



Non-PCA dimensionality reduction

- Sometimes the aim is not finding a manifold with maximal variance.
- Random projection
 - Gaussian random projection
- Multidimensional scaling (MDS)
 - Metric (typically Euclidean)
 - Non-metric
 - ISOMAP (Geodesic distances)
- T-distribution stochastic neighbor embedding (t-SNE)

41



Random projections

- Johnson-Lindenstrauss Lemma - if points in a vector space are of sufficiently high dimension, then they may be projected into a suitable lower-dimensional space in a way which approximately preserves the distances between the point.
- Thus, a random projection from N to M dimensions may be achieved via a random matrix $N \times M$.
- Gaussian random projection – uses a random distribution to generate the matrix while maintain orthogonality an normality.

42

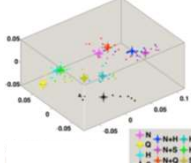





Multidimensional scaling (MDS)

- Metric MDS - Maintain pairwise distances within of high-dimensional points while embedding in a low dimensional space.
- Non metric MDS – Maintain a more general dissimilarity between points.

Embedding of neural responses to sensory stimuli (Di Lorenzo et al., 2009)

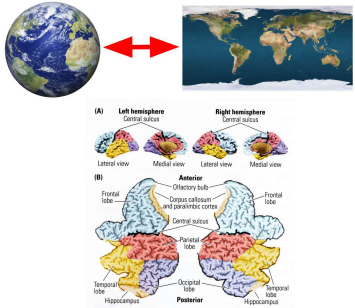


43






Manifolds



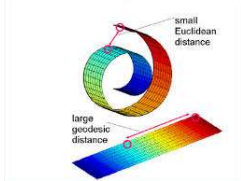
44






ISOMAP

- Non linear extension to MDS - maintain pairwise geodesic distances within of high-dimensional points while embedding in a low dimensional manifold.



45



t-SNE

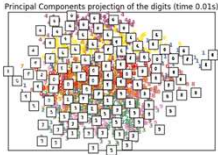
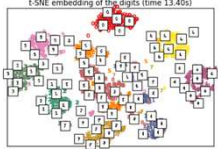
- T-distribution stochastic neighbor embedding
- Maintains local distances rather than a global feature (variance).
- Minimizes the KL divergence between the Gaussian representation of the point distances in high dimensional space with the Cauchy (1deg Student t distribution) – a leptokurtic distribution (kurtosis>0)

$$f(x; x_0, \gamma) = \frac{1}{\pi\gamma \left[1 + \left(\frac{x - x_0}{\gamma} \right)^2 \right]} = \frac{1}{\pi\gamma} \left[\frac{\gamma^2}{(x - x_0)^2 + \gamma^2} \right],$$

46



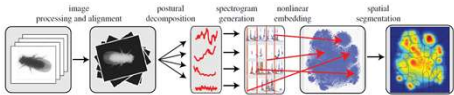
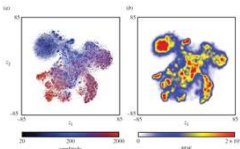
t-SNE example - digits

47



t-SNE example – animal behavior

(Berman et al, 2014)

48

16



Independent component analysis (ICA)

Not really
dimensionality reduction

Next lecture

49



Appendix

Eigenvector & Eigenvalue
Calculation

50



Transformation matrices

- Consider:

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \times \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4 \times \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$
- Square transformation matrix transforms (3,2) from its original location. Now if we were to take a multiple of (3,2)

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \times \begin{pmatrix} 6 \\ 4 \end{pmatrix} = \begin{pmatrix} 24 \\ 16 \end{pmatrix} = 4 \times \begin{pmatrix} 6 \\ 4 \end{pmatrix}$$
- Scaling a vector only changes its length and not its direction.

51



Eigenvalue problem

- The eigenvalue problem is any problem having the following form:

$$A \cdot v = \lambda \cdot v$$

$$A: n \times n \text{ matrix}$$

$$v: n \times 1 \text{ non-zero vector}$$

$$\lambda: \text{scalar}$$
- Any value of λ for which this equation has a solution is called the *eigenvalue* of A and vector v which corresponds to this value is called the *eigenvector* of A .
- In our example: $\begin{pmatrix} 3 \\ 2 \end{pmatrix}$ is an eigenvector of the square matrix A and 4 is an eigenvalue of A

52



Calculating eigenvectors & eigenvalues I

$$A \cdot v = \lambda \cdot v$$

$$A \cdot v - \lambda \cdot I \cdot v = 0$$

$$(A - \lambda \cdot I) \cdot v = 0$$

Finding the roots of $|A - \lambda \cdot I|$ will give the eigenvalues and for each of these eigenvalues there will be an eigenvector

Example ...

53



Calculating eigenvectors & eigenvalues II

- In the example: $A = \begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix}$
- So: $A - \lambda \cdot I = \begin{pmatrix} 2-\lambda & 3 \\ 2 & 1-\lambda \end{pmatrix}$

$$|A - \lambda I| = (2 - \lambda) \cdot (1 - \lambda) - 3 \cdot 2 = \lambda^2 - 3\lambda - 4 = 0$$
- Leading to: $\lambda_1 = 4 \quad \lambda_2 = -1$

54






Calculating eigenvectors & eigenvalues III

- For: $\lambda_1 = 4$

$$A - \lambda I = \begin{pmatrix} 2 - \lambda & 3 \\ 2 & 1 - \lambda \end{pmatrix} = \begin{pmatrix} -2 & 3 \\ 2 & -3 \end{pmatrix}$$

$$\begin{pmatrix} -2 & 3 \\ 2 & -3 \end{pmatrix} \cdot v_1 = 0 \rightarrow v_1 = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

- Along the same lines for $\lambda_2 = -1$ $v_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$



55






Properties of eigenvectors & eigenvalues

- All eigenvectors of a matrix are perpendicular to each other, no matter how many dimensions we have
- In practice eigenvectors are normalized to have unit length. Since the length of the eigenvectors do not affect our calculations we prefer to keep them standard by scaling them to have a length of 1.
- So, eigenvector (3,2) would be scaled by $\sqrt{3^2 + 2^2} = \sqrt{13}$



56